

 HOJA DE RUTA — CryptolA Trading Bot

Sistema Multi-Agente de Trading con IAs Locales

Servidor de IAs: 192.168.1.8

Infraestructura: Ollama en 3 instancias (puertos 11431, 11432, 11433)

Capital inicial simulado: 10.000 USDT

Activo principal: BTC/USDT (Binance)

 Estado Actual del Proyecto

El proyecto ya cuenta con una base funcional desarrollada en Python:

Archivo	Descripción	Estado
mercado.py	Conexión a Binance vía ccxt, velas OHLCV, RSI manual	✓ Funcional
noticias.py	RSS de CoinTelegraph en español	✓ Funcional
ia_trading_paso2.py	Agentes GPU0 (técnico) y GPU1 (fundamental)	✓ Funcional
ia_trading_comite.py	Comité multi-agente 3 GPUs + SQLite ORM	✓ Funcional
debate_agentes.py	Prototipo de debate entre agentes	✓ Prototipo
billetera_simulada.json	Estado de billetera en JSON	✓ Funcional
trading_bot.db	Base de datos SQLite con ~500+ registros históricos	✓ Con datos
monitoreo.sh	Script bash para monitorear GPUs	✓ Funcional

Arquitectura de IAs en el Servidor

192.168.1.8:11431 → GPU 0 → Analista Técnico (RSI, MACD, velas)

192.168.1.8:11432 → GPU 1 → Analista Fundamental (noticias, sentimiento)

192.168.1.8:11433 → GPU 2 → Gestor de Riesgos (juez final, aprueba/cancela)

 FASES DE DESARROLLO

✓ FASE 0 — Configuración del Entorno

Objetivo: Tener el proyecto limpio, organizado y con todas las dependencias instaladas.

Tareas:

- ☐ 0.1 Crear estructura de carpetas del proyecto:

```
CryptoIa/
├── src/
│   ├── mercado/
│   ├── noticias/
│   ├── agentes/
│   ├── trading/
│   └── dashboard/
├── data/
├── logs/
├── tests/
├── Fuentes_Originales/  (ya existe - no tocar)
└── requirements.txt
```

- ☐ **0.2** Crear `requirements.txt` con todas las dependencias:
 - `ccxt` (conexión a exchanges)
 - `feedparser` (RSS de noticias)
 - `requests` (llamadas HTTP a Ollama)
 - `sqlalchemy` (ORM para base de datos)
 - `python-dotenv` (variables de entorno)
 - `rich` (consola con colores y tablas)
 - `pandas` (análisis de datos)
 - `ta` (librería de indicadores técnicos)
- ☐ **0.3** Crear archivo `.env` con configuración del servidor:

```
SERVIDOR_IA=192.168.1.8
PUERTO_GPU0=11431
PUERTO_GPU1=11432
PUERTO_GPU2=11433
MODELO_GPU0=qwen2.5:7b
MODELO_GPU1=qwen2.5:3b
MODELO_GPU2=qwen2.5:3b
EXCHANGE=binance
SIMBOLO=BTC/USDT
TEMPORALIDAD=15m
CAPITAL_INICIAL=10000
```

- ☐ **0.4** Crear `config.py` centralizado que lea el `.env`
- ☐ **0.5** Verificar conectividad con el servidor `192.168.1.8` (ping + test HTTP a cada puerto Ollama)
- ☐ **0.6** Script de test de conexión: `python tests/test_conexion.py`

Criterio de éxito: `python tests/test_conexion.py` devuelve OK para los 3 puertos de Ollama.



FASE 1 — Módulo de Mercado Mejorado

Objetivo: Obtener datos de mercado más ricos con múltiples indicadores técnicos.

Tareas:

- ☐ **1.1** Refactorizar `mercado.py` → `src/mercado/binance_client.py`
 - Cambiar URLs de `localhost` a `192.168.1.8` (ya está en `.env`)
 - Agregar manejo de errores robusto con reintentos
- ☐ **1.2** Ampliar indicadores técnicos usando la librería `ta`:
 - **RSI** (14 períodos) — ya existe, mejorar
 - **MACD** (12, 26, 9) — señal de cruce
 - **Bandas de Bollinger** (20, 2) — volatilidad
 - **EMA 9 y EMA 21** — tendencia de corto plazo
 - **Volumen relativo** — comparar con promedio de 20 velas
- ☐ **1.3** Crear función `formatear_reporte_completo(datos)` que genere un reporte narrativo enriquecido para las IAs
- ☐ **1.4** Agregar soporte para múltiples temporalidades (1m, 5m, 15m, 1h)
- ☐ **1.5** Tests unitarios: `tests/test_mercado.py`

Criterio de éxito: El módulo devuelve un diccionario con precio, RSI, MACD, Bollinger, EMAs y volumen relativo.

FASE 2 — Módulo de Noticias Mejorado

Objetivo: Obtener noticias de múltiples fuentes para un análisis fundamental más robusto.

Tareas:

- ☐ **2.1** Refactorizar `noticias.py` → `src/noticias/feed_manager.py`
- ☐ **2.2** Agregar múltiples fuentes RSS de criptomonedas:
 - CoinTelegraph ES: <https://es.cointelegraph.com/rss/tag/bitcoin>
 - CoinTelegraph EN: <https://cointelegraph.com/rss/tag/bitcoin>
 - CoinDesk: <https://www.coindesk.com/arc/outboundfeeds/rss/>
 - Bitcoin Magazine: <https://bitcoinmagazine.com/feed>
- ☐ **2.3** Implementar sistema de caché para no repetir noticias ya analizadas (guardar hash del titular)
- ☐ **2.4** Función `obtener_resumen_noticias(n=3)` que devuelva los últimos N titulares concatenados
- ☐ **2.5** Filtro de relevancia: descartar noticias de más de 2 horas de antigüedad
- ☐ **2.6** Tests unitarios: `tests/test_noticias.py`

Criterio de éxito: El módulo devuelve los últimos 3 titulares relevantes de al menos 2 fuentes distintas.

FASE 3 — Comité de IAs Refactorizado

Objetivo: Refactorizar los agentes para apuntar al servidor remoto `192.168.1.8` y mejorar los prompts.

Tareas:

- ☐ **3.1** Crear `src/agentes/cliente_ollama.py` — cliente HTTP genérico para Ollama:
 - Función `consultar_ia(host, puerto, modelo, prompt, formato_json=True)`
 - Manejo de timeouts y reintentos (3 intentos con backoff)
 - Logging de cada consulta

- ☐ **3.2** Crear `src/agentes/agente_tecnico.py` (GPU 0 - puerto 11431):
 - Recibe: reporte de mercado con todos los indicadores
 - Devuelve JSON: `{"accion": "COMPRA|VENTA|ESPERAR", "confianza": 0-100, "justificacion": "..."}"`
 - Prompt mejorado con reglas de RSI, MACD y Bollinger
- ☐ **3.3** Crear `src/agentes/agente_fundamental.py` (GPU 1 - puerto 11432):
 - Recibe: resumen de últimas 3 noticias
 - Devuelve JSON: `{"impacto": "ALCISTA|BAJISTA|NEUTRAL", "intensidad": 0-100, "justificacion": "..."}"`
 - Prompt mejorado para detectar FUD vs FOMO
- ☐ **3.4** Crear `src/agentes/agente_riesgo.py` (GPU 2 - puerto 11433):
 - Recibe: veredicto técnico + veredicto fundamental + estado de la billetera
 - Devuelve JSON: `{"decision": "COMPRA|VENTA|ESPERAR", "stop_loss_pct": 2.5, "take_profit_pct": 5.0, "motivo": "..."}"`
 - Prompt con reglas de gestión de capital (nunca arriesgar más del 2% por operación)
- ☐ **3.5** Crear `src/agentes/comite.py` — orquestador del debate:
 - Ejecuta los 3 agentes en secuencia
 - Implementa lógica de consenso: si GPU0 y GPU1 coinciden, GPU2 aprueba automáticamente
 - Registra el debate completo en la base de datos
- ☐ **3.6** Tests de integración: `tests/test_comite.py`

Criterio de éxito: El comité devuelve una decisión estructurada en menos de 30 segundos.



FASE 4 — Motor de Trading con Gestión de Riesgo

Objetivo: Implementar un motor de trading robusto con stop-loss, take-profit y gestión de capital.

Tareas:

- ☐ **4.1** Crear `src/trading/billetera.py`:
 - Migrar de JSON a SQLite (tabla `billetera`)
 - Historial de saldos con timestamps
 - Soporte para múltiples activos (BTC, ETH, etc.)
- ☐ **4.2** Crear `src/trading/gestor_ordenes.py`:
 - `ejecutar_compra(precio, porcentaje_capital=100)` — compra con % del capital disponible
 - `ejecutar_venta(precio, motivo)` — venta con registro del motivo
 - `verificar_stop_loss(precio_actual)` — cierra posición si cae X%
 - `verificar_take_profit(precio_actual)` — cierra posición si sube X%
- ☐ **4.3** Crear `src/trading/motor_principal.py` — bucle principal 24/7:
 - Ciclo cada 60 segundos (configurable)
 - Integra: mercado → noticias → comité → órdenes
 - Manejo de excepciones para no detener el bot
 - Logging detallado de cada ciclo
- ☐ **4.4** Implementar reglas de gestión de riesgo:
 - **Stop-Loss dinámico:** -2.5% del precio de compra (configurable)
 - **Take-Profit dinámico:** +5% del precio de compra (configurable)

- **Máximo de operaciones por día:** 5 (para evitar overtrading)
- **Cooldown entre operaciones:** 15 minutos mínimo
- ☐ **4.5** Crear `src/trading/base_datos.py`:
 - Migrar el esquema SQLite existente
 - Agregar tabla `ciclos_bot` para registrar cada iteración
 - Agregar tabla `configuracion` para parámetros dinámicos
- ☐ **4.6** Tests de simulación: `tests/test_motor.py` con datos históricos del `trading_bot.db`

Criterio de éxito: El bot corre 1 hora sin errores, registra todos los ciclos en la DB y respeta stop-loss/take-profit.



FASE 5 — Dashboard de Monitoreo y Reportes

Objetivo: Tener visibilidad en tiempo real del estado del bot y su rendimiento.

Tareas:

- ☐ **5.1** Crear `src/dashboard/monitor_consola.py` usando la librería `rich`:
 - Tabla en tiempo real con: precio BTC, RSI, decisión del comité, saldo, P&L
 - Historial de las últimas 10 operaciones
 - Estado de cada GPU (conectada/desconectada)
- ☐ **5.2** Crear `src/dashboard/reporte_diario.py`:
 - Genera resumen diario en texto: operaciones, rendimiento, win rate
 - Exporta a CSV para análisis en Excel
- ☐ **5.3** Crear `src/dashboard/analisis_historico.py`:
 - Analiza los datos del `trading_bot.db` existente
 - Calcula métricas: Sharpe ratio, max drawdown, win rate
 - Identifica patrones: ¿en qué condiciones de RSI el bot gana más?
- ☐ **5.4** Script de inicio unificado `run_bot.py`:
 - Verifica conectividad con el servidor antes de arrancar
 - Muestra dashboard en consola
 - Maneja señales de sistema (Ctrl+C limpio)
- ☐ **5.5** Crear `monitoreo_windows.bat` (equivalente al `monitoreo.sh` para Windows)

Criterio de éxito: `python run_bot.py` arranca el sistema completo con dashboard visible.



FASE 6 — Mejoras Avanzadas (Futuro)

Objetivo: Evolucionar el sistema hacia estrategias más sofisticadas.

Ideas para el futuro:

- ☐ **6.1** Agregar un **4to agente** (GPU 3 si está disponible): Analista de Sentimiento en Twitter/X
- ☐ **6.2** Implementar **backtesting** con datos históricos de Binance (últimos 6 meses)
- ☐ **6.3** Soporte para **múltiples activos** simultáneos (ETH, SOL, BNB)
- ☐ **6.4 Auto-ajuste de parámetros:** el bot aprende de sus errores y ajusta stop-loss/take-profit
- ☐ **6.5 Interfaz web** con Flask/FastAPI para monitoreo remoto desde el navegador

- ☐ **6.6 Alertas por Telegram:** notificaciones cuando se ejecuta una compra/venta
- ☐ **6.7** Migrar la base de datos de SQLite a **PostgreSQL** para mayor robustez
- ☐ **6.8** Implementar **paper trading** en Binance Testnet antes de operar con dinero real

 RESUMEN DE PROGRESO

Fase	Descripción	Estado
FASE 0	Configuración del entorno	 Pendiente
FASE 1	Módulo de mercado mejorado	 Pendiente
FASE 2	Módulo de noticias mejorado	 Pendiente
FASE 3	Comité de IAs refactorizado	 Pendiente
FASE 4	Motor de trading con gestión de riesgo	 Pendiente
FASE 5	Dashboard de monitoreo	 Pendiente
FASE 6	Mejoras avanzadas	 Futuro

 Decisiones Técnicas Clave

Decisión	Elección	Motivo
Conexión a IAs	HTTP REST a 192.168.1.8:1143X	Ollama ya corriendo en el servidor
Base de datos	SQLite → migrable a PostgreSQL	Simple para empezar, escalable
Exchange	Binance vía ccxt	Mayor liquidez, API gratuita
Indicadores	Librería ta	Más completa que cálculo manual
Configuración	.env + python-dotenv	Seguro, no commitear credenciales
Logging	rich + archivos .log	Visual en consola + persistente

 Notas Importantes

- NUNCA** commitear el archivo **.env** al repositorio (ya está en **.gitignore**)
- El bot opera en **modo simulación** hasta que se valide en producción
- Los modelos de Ollama en el servidor deben estar **precargados** antes de iniciar el bot
- El servidor **192.168.1.8** debe ser accesible desde la máquina de desarrollo
- Respetar los **rate limits** de Binance (máximo 1200 requests/minuto)

Última actualización: Octubre 2026

Próximo paso: Comenzar con FASE 0 — Configuración del entorno